

June 13, 2026 at 09:46

## 1. Countdown.

This is a program written in Donald Knuth’s *literate programming* paradigm. It opens a window on the X Window System and displays a live countdown — days, hours, minutes, and seconds — to a fixed future instant. The default target is

2026-08-19 08:00:00

(the morning of 19 August 2026), but any target may be supplied on the command line. The user starts the countdown with the keyboard or the mouse, may pause and resume it, and quits when finished.

**2. Literate programming.** Donald Knuth introduced literate programming in 1984 as a way of writing software meant to be read by human beings first and executed by computers second. Rather than annotating code with comments, a literate program interweaves prose and code in a single source document (conventionally given the extension `.w`). The prose explains the *why*; the code expresses the *how*. Two companion tools separate them at build time: `ctangle` extracts the compilable C source, and `cweave` produces the typeset documentation you are now reading. In this project both are driven through the `cweb.sh` helper script:

```
./cweb.sh ctangle countdown.w    ./cweb.sh cweave countdown.w
```

**3. A note on module size.** By project convention every numbered module (the unit `cweave` typesets as one “section”) contains fewer than twenty-four lines of code. Long constructions are therefore split into several named sections that are stitched together at tangle time. This keeps each idea small enough to be grasped whole.

**4. Program structure.** The tangled output is assembled in the order below. Each named section is elaborated in its own module further on.

- ⟨Header files 5⟩
- ⟨Constants 6⟩
- ⟨The application state *App* 7⟩
- ⟨Parse the target time 8⟩
- ⟨Compute the remaining seconds 10⟩
- ⟨Format a duration 11⟩
- ⟨Open the display 12⟩
- ⟨Create the window 13⟩
- ⟨Create the graphics context 14⟩
- ⟨Render the display 15⟩
- ⟨Handle one X event 17⟩
- ⟨The event-and-timer loop 20⟩
- ⟨The *main* function 24⟩

**5. Header files.** The program draws on three families of declarations: the C standard library, the POSIX operating-system interface, and the Xlib client library for the X Window System.

⟨Header files 5⟩ ≡

```
#include <stdio.h>    /* printf, fprintf, snprintf, sscanf */
#include <stdlib.h>    /* exit */
#include <string.h>    /* memset, strlen */
#include <time.h>      /* time, mktime, localtime, clock_gettime */
#include <unistd.h>    /* POSIX symbolic constants */
#include <sys/select.h> /* select, fd_set, FD_SET */
#include <X11/Xlib.h>  /* the core Xlib client interface */
#include <X11/Xutil.h> /* XFontStruct and window-manager hints */
```

This code is used in section 4.

**6. Constants.** A handful of compile-time constants fix the default target instant, the geometry of the window, and the X11 font we ask the server to load. The font `10x20` is a fixed-width bitmap font present on virtually every X server; if it is unavailable we fall back to the server default.

```
#define DEFAULT_TARGET "2026-08-19_08:00:00" /* 19 August 2026, 8:00 AM */
#define WIN_W 520 /* window width in pixels */
#define WIN_H 220 /* window height in pixels */
#define FONT_NAME "10x20" /* preferred fixed-width font */
#define TICK_USEC 250000 /* event-loop wake-up interval, microseconds */
```

⟨Constants 6⟩ ≡ /\* The constants above are expressed as preprocessor macros so that the tangled  
C source carries no separate definitions module. \*/

This code is used in section 4.

**7. The application state.** All mutable state lives in one *App* structure that is threaded through the drawing and event-handling routines. Keeping it in a single record (rather than in file-scope globals) makes the data flow explicit and the routines easy to test in isolation.

⟨The application state **App 7**⟩ ≡

```
typedef struct {  
    Display * dpy;    /* connection to the X server */  
    Window win;      /* our top-level window */  
    GC gc;           /* graphics context for text */  
    XFontStruct * font; /* loaded font, or Λ */  
    int screen;        /* default screen number */  
    time_t target;     /* the instant we count down to */  
    int running;       /* 1 while counting, 0 while paused or armed */  
    int started;       /* 1 once the user has begun the countdown */  
    long frozen;       /* remaining seconds captured at the last pause */  
} App;
```

This code is used in section 4.

**8. Parsing the target time.** The target is given as the calendar string "YYYY-MM-DD HH:MM:SS" in *local* time. We decompose it with *sscanf*, populate a **struct** *tm*, and convert it to a **time\_t** with the POSIX function *mktime*. Setting *tm\_isdst* to  $-1$  asks *mktime* to determine for itself whether daylight saving time is in effect on that date.

⟨Parse the target time 8⟩ ≡

```
static time_t parse_target(const char *s)
{
    struct tm tm;
    int yr, mo, dy, hr, mi, se;
    memset(&tm, 0, sizeof tm);
    if (sscanf(s, "%d-%d-%d %d:%d:%d", &yr, &mo, &dy, &hr, &mi, &se) ≠ 6) {
        fprintf(stderr, "countdown: bad target \"%s\" \"(want YYYY-MM-DD HH:MM:SS)\"\\n", s);
        exit(2);
    }
    ⟨Fill the broken-down time and convert it 9⟩
}
```

This code is used in section 4.

**9.** The fields of **struct** *tm* count years from 1900 and months from zero, so we adjust accordingly before calling *mktime*. A return value of  $(time_t)(-1)$  signals an unrepresentable date.

⟨Fill the broken-down time and convert it 9⟩ ≡

```
{
    time_t t;
    tm.tm_year = yr - 1900;
    tm.tm_mon = mo - 1;
    tm.tm_mday = dy;
    tm.tm_hour = hr;
    tm.tm_min = mi;
    tm.tm_sec = se;
    tm.tm_isdst = -1; /* let mktime decide about DST */
    t = mktime(&tm);
    if (t ≡ (time_t)(-1)) {
        fprintf(stderr, "countdown: target is not a valid date\\n");
        exit(2);
    }
    return t;
}
```

This code is used in section 8.

**10. Computing the remaining seconds.** The number of whole seconds still to run is the difference between the target and the current instant, clamped at zero so the display never goes negative. We read the wall clock with the POSIX *time* call.

⟨ Compute the remaining seconds 10 ⟩  $\equiv$

```
static long remaining_secs(time_t target)
{
    time_t now = time(&);
    double d = difftime(target, now);
    return d > 0.0 ? (long) d : 0_L;
}
```

See also section 19.

This code is used in section 4.

**11. Formatting a duration.** A count of seconds is rendered as days plus a 24-hour clock. The result is written into a caller-supplied buffer with the bounds-checked *snprintf*, so no overflow is possible regardless of the magnitude.

⟨Format a duration 11⟩ ≡

```
static void fmt_duration(long secs, char *buf, size_t n)
{
    long d = secs / 86400;
    long r = secs % 86400;
    long h = r / 3600;
    long m = (r % 3600) / 60;
    long s = r % 60;
    snprintf(buf, n, "%ld_d_%02ld:%02ld:%02ld", d, h, m, s);
}
```

This code is used in section 4.

**12. Opening the display.** Connecting to the X server is the first Xlib act. *XOpenDisplay*( $\Lambda$ ) honours the `DISPLAY` environment variable, which POSIX makes available through *getenv* inside Xlib. A null return means no server could be reached, which is fatal.

⟨ Open the display 12 ⟩  $\equiv$

```
static void open_display(App *a)
{
    a->dpy = XOpenDisplay( $\Lambda$ );
    if (a->dpy  $\equiv$   $\Lambda$ ) {
        fprintf(stderr,
            "countdown: cannot open X display " " (is DISPLAY set and an X server running?)\n");
        exit(1);
    }
    a->screen = DefaultScreen(a->dpy);
}
```

This code is used in section 4.

**13. Creating the window.** We ask for a simple top-level window with a white background and a thin black border, give it a title for the window manager, and declare which event categories we wish to receive: exposures (redraw requests), key and button presses, and structure-change notifications (resizing).

⟨ Create the window 13 ⟩ ≡

```
static void create_window(App *a)
{
    unsigned long white = WhitePixel(a-dpy, a-screen);
    unsigned long black = BlackPixel(a-dpy, a-screen);
    a-win = XCreateSimpleWindow(a-dpy, RootWindow(a-dpy, a-screen), 0, 0, WIN_W, WIN_H, 2, black,
                               white);
    XStoreName(a-dpy, a-win, "Countdown");
    XSelectInput(a-dpy, a-win, ExposureMask | KeyPressMask | ButtonPressMask | StructureNotifyMask);
    XMapWindow(a-dpy, a-win);
}
```

This code is used in section 4.

**14. Creating the graphics context.** The graphics context (GC) holds the drawing attributes — foreground colour and font — used by every text-drawing call. We load the preferred font and, if it is present, bind it to the GC; otherwise Xlib’s default font is used.

⟨ Create the graphics context 14 ⟩ ≡

```
static void create_gc(App *a)
{
    a->gc = XCreateGC(a->dpy, a->win, 0, Λ);
    XSetForeground(a->dpy, a->gc, BlackPixel(a->dpy, a->screen));
    a->font = XLoadQueryFont(a->dpy, FONT_NAME);
    if (a->font ≠ Λ) XSetFont(a->dpy, a->gc, a->font->fid);
}
```

This code is used in section 4.

**15. Rendering the display.** Each repaint clears the window, prints the target instant on the first line, the remaining time in large digits on the second, and a status and hint on the lower lines. When the countdown has not yet started, or is paused, the figure is taken from *frozen* rather than recomputed, so it holds still.

```

⟨ Render the display 15 ⟩ ≡
static void render(App *a)
{
    static const char *hint = "[Space]_or_click:_start/pause_[]:_quit";
    char hdr[80], big[48], status[64];
    long secs = (a-running) ? remaining_secs(a-target) : a-frozen;
    ⟨ Build the header and status strings 16 ⟩
    XClearWindow(a-dpy, a-win);
    XDrawString(a-dpy, a-win, a-gc, 24, 40, hdr, strlen(hdr));
    XDrawString(a-dpy, a-win, a-gc, 24, 110, big, strlen(big));
    XDrawString(a-dpy, a-win, a-gc, 24, 150, status, strlen(status));
    XDrawString(a-dpy, a-win, a-gc, 24, 195, hint, strlen(hint));
    XFlush(a-dpy);
}

```

This code is used in section 4.

**16.** The header restates the target in a human-readable form produced by *localtime* and *strftime*. The status line names the current phase and, when the countdown has reached zero, announces that the moment has arrived.

```

⟨ Build the header and status strings 16 ⟩ ≡
{
    struct tm *lt = localtime(&a-target);
    strftime(hdr, sizeof hdr, "Target:_%a_%d_%b_%Y_%H:%M:%S", lt);
    fmt_duration(secs, big, sizeof big);
    if (secs == 0 ^ a-started) snprintf(status, sizeof status, "Status:_THE_MOMENT_HAS_ARRIVED");
    else if (!a-started) snprintf(status, sizeof status, "Status:_ready_--_press_Space_to_start");
    else snprintf(status, sizeof status, "Status:_%s", a-running ? "running" : "paused");
}

```

This code is used in section 15.

**17. Handling one X event.** The dispatcher reacts to the four event categories we subscribed to. An *Expose* or *ConfigureNotify* simply triggers a repaint. A key press of **q** or **Escape** requests quit (by returning 0); the space bar toggles the run state. A mouse click anywhere also toggles the run state. Any other event leaves the loop running.

```

⟨Handle one X event 17⟩ ≡
static int handle_event(App *a, XEvent *ev)
{
    switch (ev->type) {
        case Expose: case ConfigureNotify: render(a);
            break;
        case KeyPress: ⟨React to a key press 18⟩
            break;
        case ButtonPress: toggle_run(a);
            render(a);
            break;
    }
    return 1;    /* keep looping */
}

```

This code is used in section 4.

**18.** A key press is decoded to a *KeySym*. We treat **q** and the escape key as quit requests, and the space bar as the run/pause toggle.

```

⟨React to a key press 18⟩ ≡
{
    KeySym k = XLookupKeysym(&ev->xkey, 0);
    if (k ≡ XK_q ∨ k ≡ XK_Escape) return 0;    /* signal the caller to quit */
    if (k ≡ XK_space) {
        toggle_run(a);
        render(a);
    }
}

```

This code is used in section 17.

**19.** Toggling the run state has three cases. Starting for the first time arms the countdown and marks it started. Pausing captures the current remaining seconds into *frozen* so the display holds. Resuming simply sets *running* again.

```

⟨Compute the remaining seconds 10⟩ +≡
static void toggle_run(App *a)
{
    if (¬a->started) {
        a->started = 1;
        a->running = 1;
    }
    else if (a->running) {
        a->frozen = remaining_secs(a->target);
        a->running = 0;
    }
    else {
        a->running = 1;
    }
}

```

**20. The event-and-timer loop.** A countdown must both react to user input *and* tick forward on its own. Pure *XNextEvent* would block until the next event, so the clock would freeze whenever the user sat still. Instead we multiplex the X connection's file descriptor against a timer using the POSIX *select* call: *select* returns either when an event is readable or when the timeout lapses, and in the latter case we repaint to advance the clock.

```

⟨ The event-and-timer loop 20 ⟩ ≡
static void run_loop(App *a)
{
    int xfd = ConnectionNumber(a-dpy);
    for ( ; ; ) {
        ⟨ Drain all pending X events; quit if asked 21 ⟩
        ⟨ Wait up to one tick for the next event 22 ⟩
        if (a-running) render(a);
    }
}

```

This code is used in section 4.

**21.** Before sleeping we must process every event already queued, because *select* reports readability only for bytes still in the socket, not for events Xlib has already buffered. *XPending* reports the buffered count.

```

⟨ Drain all pending X events; quit if asked 21 ⟩ ≡
while (XPending(a-dpy) > 0) {
    XEvent ev;
    XNextEvent(a-dpy, &ev);
    if (!handle_event(a, &ev)) {
        ⟨ Tear down and exit cleanly 23 ⟩
    }
}

```

This code is used in section 20.

**22.** The timeout is rebuilt on every iteration because *select* may modify the *timeval* it is given. A quarter-second tick keeps the seconds digit visually crisp without busy-waiting. An interruption by a signal (EINTR) is harmless: the loop simply comes round again.

```

⟨ Wait up to one tick for the next event 22 ⟩ ≡
{
    fd_set rfd;
    struct timeval tv;
    FD_ZERO(&rfd);
    FD_SET(xfd, &rfd);
    tv.tv_sec = 0;
    tv.tv_usec = TICK_USEC;
    select(xfd + 1, &rfd, Λ, Λ, &tv);
}

```

This code is used in section 20.

**23.** Quitting releases the resources we acquired, in the reverse order of acquisition, then leaves the program.

⟨Tear down and exit cleanly 23⟩ ≡

```
{  
  if (a-font) XFreeFont(a-dpy, a-font);  
  XFreeGC(a-dpy, a-gc);  
  XDestroyWindow(a-dpy, a-win);  
  XCloseDisplay(a-dpy);  
  exit(0);  
}
```

This code is used in section 21.

**24. The main function.** *main* selects the target (command-line argument or default), initialises the **App** record, brings up the GUI, paints once, and enters the loop.

⟨The *main* function 24⟩ ≡

```
int main(int argc, char **argv)
{
    App a;
    const char *spec = (argc > 1) ? argv[1] : DEFAULT_TARGET;
    memset(&a, 0, sizeof a);
    a.target = parse_target(spec);
    a.frozen = remaining_secs(a.target);
    ⟨Start the GUI and run 25⟩
    return 0;
}
```

This code is used in section 4.

**25.** The startup sequence opens the server connection, creates the window and its graphics context, draws the initial frame, and hands control to the loop, which never returns normally.

⟨Start the GUI and run 25⟩ ≡

```
{
    open_display(&a);
    create_window(&a);
    create_gc(&a);
    render(&a);
    run_loop(&a);
}
```

This code is used in section 24.

**26. POSIX system services.** This program rests on a small, deliberately chosen set of POSIX operating system services. They are gathered and explained here, and each is also entered in the index under the heading *system call*, so that a reader auditing the program's interaction with the operating system can find every one of them from a single place.

**27. *time(2)* — get the current calendar time.** *time*( $\Lambda$ ) returns the number of seconds since the Unix epoch (1970-01-01T00:00:00 UTC) as a **time\_t**. We use it in *remaining\_secs* to learn how far the present is from the target. It cannot fail when given a null argument on a conforming system.

**28. *mktime(3)* — convert broken-down local time to **time\_t**.** Given a **struct tm** expressed in local time, *mktime* returns the corresponding **time\_t**, normalising out-of-range fields and consulting the TZ timezone database to apply the correct UTC offset and daylight-saving rule. We call it once, in *parse\_target*, to turn the user's calendar string into an absolute instant.

**29. *localtime(3)* — convert **time\_t** to broken-down local time.** *localtime* is the inverse of *mktime*: it explodes a **time\_t** into a **struct tm** in the local zone, which *strftime* then formats for the header line. The returned pointer addresses a static buffer that may be overwritten by later calls; we consume it immediately.

**30. *select(2)* — synchronous I/O multiplexing with a timeout.** *select* watches a set of file descriptors and returns when one becomes ready or when a timeout expires. We watch the single descriptor returned by *ConnectionNumber* (the socket to the X server) with a quarter-second timeout, so the program wakes either to service user input or to advance the clock — never busy-waiting, never sleeping through an event. The *fd\_set* is populated with the POSIX macros FD\_ZERO and FD\_SET.

**31. *getenv(3)* — look up an environment variable.** We do not call *getenv* directly, but Xlib calls it on our behalf inside *XOpenDisplay* to read the DISPLAY variable, which names the X server to contact. It is documented here for completeness because it governs which display the program attaches to.

**32. Index.** The cross-reference index is generated automatically by **cweave**. Identifiers appear in *italic*, named sections by the module in which they are defined, and the POSIX system services under the heading *system call* as described above.

*a*: [12](#), [13](#), [14](#), [15](#), [17](#), [19](#), [20](#), [24](#).  
**App**: [7](#), [12](#), [13](#), [14](#), [15](#), [17](#), [19](#), [20](#), [24](#).  
*argc*: [24](#).  
*argv*: [24](#).  
*big*: [15](#), [16](#).  
*black*: [13](#).  
*BlackPixel*: [13](#), [14](#).  
*buf*: [11](#).  
*ButtonPress*: [17](#).  
*ButtonPressMask*: [13](#).  
*clock\_gettime*: [5](#).  
*ConfigureNotify*: [17](#).  
*ConnectionNumber*: [20](#), [30](#).  
*create\_gc*: [14](#), [25](#).  
*create\_window*: [13](#), [25](#).  
*d*: [10](#), [11](#).  
**DEFAULT\_TARGET**: [6](#), [24](#).  
*DefaultScreen*: [12](#).  
*difftime*: [10](#).  
**DISPLAY**: [12](#), [31](#).  
*Display*: [7](#).  
*dpy*: [7](#), [12](#), [13](#), [14](#), [15](#), [20](#), [21](#), [23](#).  
*dy*: [8](#), [9](#).  
**EINTR**: [22](#).  
*ev*: [17](#), [18](#), [21](#).  
*exit*: [5](#), [8](#), [9](#), [12](#), [23](#).  
*Expose*: [17](#).  
*ExposureMask*: [13](#).  
*fd\_set*: [5](#), [22](#), [30](#).  
**FD\_SET**: [5](#), [22](#), [30](#).  
**FD\_ZERO**: [22](#), [30](#).  
*fid*: [14](#).  
*fmt\_duration*: [11](#), [16](#).  
*font*: [7](#), [14](#), [23](#).  
**FONT\_NAME**: [6](#), [14](#).  
*fprintf*: [5](#), [8](#), [9](#), [12](#).  
*frozen*: [7](#), [15](#), [19](#), [24](#).  
*gc*: [7](#), [14](#), [15](#), [23](#).  
**GC**: [7](#).  
*getenv*: [12](#), [31](#).  
*h*: [11](#).  
*handle\_event*: [17](#), [21](#).  
*hdr*: [15](#), [16](#).  
*hint*: [15](#).  
*hr*: [8](#), [9](#).  
*KeyPress*: [17](#).  
*KeyPressMask*: [13](#).  
*KeySym*: [18](#).  
*localtime*: [5](#), [16](#), [29](#).  
*lt*: [16](#).  
*m*: [11](#).  
*main*: [24](#).  
*memset*: [5](#), [8](#), [24](#).  
*mi*: [8](#), [9](#).  
*mktime*: [5](#), [8](#), [9](#), [28](#), [29](#).  
*mo*: [8](#), [9](#).  
*n*: [11](#).  
*now*: [10](#).  
*open\_display*: [12](#), [25](#).  
*parse\_target*: [8](#), [24](#), [28](#).  
*printf*: [5](#).  
*r*: [11](#).  
*remaining\_secs*: [10](#), [15](#), [19](#), [24](#), [27](#).  
*render*: [15](#), [17](#), [18](#), [20](#), [25](#).  
*rfds*: [22](#).  
*RootWindow*: [13](#).  
*run\_loop*: [20](#), [25](#).  
*running*: [7](#), [15](#), [16](#), [19](#), [20](#).  
*s*: [8](#), [11](#).  
*screen*: [7](#), [12](#), [13](#), [14](#).  
*se*: [8](#), [9](#).  
*secs*: [11](#), [15](#), [16](#).  
*select*: [5](#), [20](#), [21](#), [22](#), [30](#).  
*snprintf*: [5](#), [11](#), [16](#).  
*spec*: [24](#).  
*sscanf*: [5](#), [8](#).  
*started*: [7](#), [16](#), [19](#).  
*status*: [15](#), [16](#).  
*stderr*: [8](#), [9](#), [12](#).  
*strftime*: [16](#), [29](#).  
*strlen*: [5](#), [15](#).  
*StructureNotifyMask*: [13](#).  
system call: **FD\_SET**: [30](#).  
system call: **FD\_ZERO**: [30](#).  
system call: **getenv**: [31](#).  
system call: **localtime**: [29](#).  
system call: **mktime**: [28](#).  
system call: **select**: [30](#).  
system call: **time**: [27](#).  
*t*: [9](#).  
*target*: [7](#), [10](#), [15](#), [16](#), [19](#), [24](#).  
**TICK\_USEC**: [6](#), [22](#).  
*time*: [5](#), [10](#), [27](#).  
*timeval*: [22](#).  
*tm*: [8](#), [9](#), [16](#), [28](#), [29](#).  
*tm\_hour*: [9](#).  
*tm\_isdst*: [8](#), [9](#).  
*tm\_mday*: [9](#).

*tm\_min*: 9.  
*tm\_mon*: 9.  
*tm\_sec*: 9.  
*tm\_year*: 9.  
*toggle\_run*: 17, 18, 19.  
*tv*: 22.  
*tv\_sec*: 22.  
*tv\_usec*: 22.  
*type*: 17.  
TZ: 28.  
*white*: 13.  
*WhitePixel*: 13.  
*win*: 7, 13, 14, 15, 23.  
WIN\_H: 6, 13.  
WIN\_W: 6, 13.  
*Window*: 7.  
*XClearWindow*: 15.  
*XCloseDisplay*: 23.  
*XCreateGC*: 14.  
*XCreateSimpleWindow*: 13.  
*XDestroyWindow*: 23.  
*XDrawString*: 15.  
*XEvent*: 17, 21.  
*xfd*: 20, 22.  
*XFlush*: 15.  
*XFontStruct*: 5, 7.  
*XFreeFont*: 23.  
*XFreeGC*: 23.  
*XK\_Escape*: 18.  
*XK\_q*: 18.  
*XK\_space*: 18.  
*xkey*: 18.  
*XLoadQueryFont*: 14.  
*XLookupKeysym*: 18.  
*XMapWindow*: 13.  
*XNextEvent*: 20, 21.  
*XOpenDisplay*: 12, 31.  
*XPending*: 21.  
*XSelectInput*: 13.  
*XSetFont*: 14.  
*XSetForeground*: 14.  
*XStoreName*: 13.  
*yr*: 8, 9.

⟨ Build the header and status strings 16 ⟩    Used in section 15.  
⟨ Compute the remaining seconds 10, 19 ⟩    Used in section 4.  
⟨ Constants 6 ⟩    Used in section 4.  
⟨ Create the graphics context 14 ⟩    Used in section 4.  
⟨ Create the window 13 ⟩    Used in section 4.  
⟨ Drain all pending X events; quit if asked 21 ⟩    Used in section 20.  
⟨ Fill the broken-down time and convert it 9 ⟩    Used in section 8.  
⟨ Format a duration 11 ⟩    Used in section 4.  
⟨ Handle one X event 17 ⟩    Used in section 4.  
⟨ Header files 5 ⟩    Used in section 4.  
⟨ Open the display 12 ⟩    Used in section 4.  
⟨ Parse the target time 8 ⟩    Used in section 4.  
⟨ React to a key press 18 ⟩    Used in section 17.  
⟨ Render the display 15 ⟩    Used in section 4.  
⟨ Start the GUI and run 25 ⟩    Used in section 24.  
⟨ Tear down and exit cleanly 23 ⟩    Used in section 21.  
⟨ The application state **App** 7 ⟩    Used in section 4.  
⟨ The event-and-timer loop 20 ⟩    Used in section 4.  
⟨ The *main* function 24 ⟩    Used in section 4.  
⟨ Wait up to one tick for the next event 22 ⟩    Used in section 20.

# COUNTDOWN

|                                       | Section            | Page |
|---------------------------------------|--------------------|------|
| Countdown .....                       | <a href="#">1</a>  | 1    |
| Header files .....                    | <a href="#">5</a>  | 2    |
| Constants .....                       | <a href="#">6</a>  | 3    |
| The application state .....           | <a href="#">7</a>  | 4    |
| Parsing the target time .....         | <a href="#">8</a>  | 5    |
| Computing the remaining seconds ..... | <a href="#">10</a> | 6    |
| Formatting a duration .....           | <a href="#">11</a> | 7    |
| Opening the display .....             | <a href="#">12</a> | 8    |
| Creating the window .....             | <a href="#">13</a> | 9    |
| Creating the graphics context .....   | <a href="#">14</a> | 10   |
| Rendering the display .....           | <a href="#">15</a> | 11   |
| Handling one X event .....            | <a href="#">17</a> | 12   |
| The event-and-timer loop .....        | <a href="#">20</a> | 13   |
| The main function .....               | <a href="#">24</a> | 15   |
| POSIX system services .....           | <a href="#">26</a> | 16   |
| Index .....                           | <a href="#">32</a> | 17   |